Inteligencia Artificial Generativa Avanzada y Sistemas Multi-Agente

Apuntes de clase — UTDT
Prof. Francisco Traversaro

Clase 1: De vectores a un LLM — Fundamentos matemáticos

Todo LLM se apoya en una cadena de bloques: vectores/embeddings → red neuronal → atención → Transformer → LLM. En deep learning, cada dato (token, imagen) se codifica como un vector en dimensión alta, lo que se llama representación distribuida. Los **embeddings** son vectores aprendidos por palabra cuya geometría captura significado: la famosa analogía vec(rey) − vec(hombre) + vec(mujer) ≈ vec(reina) es un ejemplo clásico. La operación fundamental para medir qué tan alineados están dos vectores es el **producto interno**, que en atención determina el peso de un token sobre otro (escalado por $\sqrt{d}$ y normalizado con softmax).

Una **capa lineal** es simplemente una matriz $W \in \mathbb{R}^{m \times n}$ que aplica m productos internos en paralelo. Componer varias capas lineales equivale a multiplicar matrices, lo cual domina el costo del entrenamiento y justifica el uso de GPUs. Sin embargo, apilar capas lineales sin no linealidad no agrega expresividad: la composición de funciones afines sigue siendo afín. Por eso cada capa incluye una función de activación ϕ (ReLU, sigmoide, tanh) que rompe la linealidad. El **teorema de aproximación universal** (Cybenko 1989) garantiza que una red con una capa oculta y activación no polinomial puede aproximar cualquier función continua sobre un compacto, aunque no dice nada sobre cuántas neuronas se necesitan ni si el entrenamiento las encuentra.

Entrenar una red es resolver un problema de optimización: minimizar una función de pérdida $L(\theta)$ sobre los parámetros. Para regresión se usa MSE, para clasificación entropía cruzada con softmax. El **gradiente** $\nabla_\theta L$ apunta en la dirección de máximo ascenso, y la regla de actualización $\theta \leftarrow \theta - \eta \nabla_\theta L$ lo usa para descender. En la práctica se usa **Adam** (momentum + escala adaptativa), y el gradiente se estima sobre minibatches (SGD). **Backpropagation** calcula el gradiente completo en una sola pasada hacia atrás usando la regla de la cadena sobre el grafo de cómputo, con costo del mismo orden que el forward.

El laboratorio de esta clase construye un motor de diferenciación automática en modo reverso (**micrograd**) en unas 100 líneas de Python. Cada nodo (`Value`) guarda su valor, su gradiente y una closure que propaga el gradiente hacia sus padres. Sobre esa base se construyen neuronas, capas y un MLP. El loop de entrenamiento (forward → loss → zero_grad → backward → update) es conceptualmente el mismo con que se entrena un LLM. Punto clave: los gradientes se **acumulan** (`+=`), por eso `zero_grad()` es imprescindible en cada paso.

Clase 2: Attention is All You Need — Transformers

El paper de Vaswani et al. (NeurIPS 2017) introdujo el Transformer: una arquitectura basada enteramente en atención, sin recurrencia ni convoluciones. El primer paso es convertir texto en tokens. Hay tres granularidades (carácter, subpalabra, palabra), y el trade-off central es tamaño del vocabulario V vs largo de la secuencia T. La tokenización por subpalabras (BPE) busca el equilibrio.

El modelo más simple es el **bigrama**, que predice el próximo carácter mirando únicamente el actual. Su techo es claro: no tiene contexto. Un primer intento de darle memoria es promediar los embeddings de todos los tokens anteriores (bag of words del pasado), pero esto descarta el orden y la relevancia. La **self-attention** resuelve esto: cada token produce tres proyecciones lineales — query ("qué busco"), key ("qué ofrezco") y value ("qué aporto si me eligen") — y la afinidad entre tokens se calcula como el producto interno entre queries y keys, escalado por $1/\sqrt{d_k}$ para evitar saturación del softmax. La fórmula resultante es $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top/\sqrt{d_k} + \text{máscara}) \cdot V$. Como la atención es ciega al orden, se suma un **positional encoding** a la entrada.

Una sola cabeza de atención captura un solo tipo de relación. **Multi-head attention** ejecuta H cabezas independientes en subespacios de tamaño C/H, las concatena y aplica una proyección W_O. El costo en parámetros es el mismo que una cabeza grande, pero la expresividad es mucho mayor. El bloque Transformer completo combina multi-head attention (que **comunica** tokens) con una FFN (que **procesa** cada token individualmente), usando conexiones residuales ($x \leftarrow x + \text{Sublayer}(x)$) para combatir el desvanecimiento del gradiente y LayerNorm para estabilizar las activaciones. Se apilan N de estos bloques: BERT usa 12, GPT-3 usa 96.

Existen tres variantes: **encoder-only** (BERT, bidireccional, para comprensión), **decoder-only** (GPT, causal con máscara, para generación) y **encoder-decoder** (T5, el paper original, para secuencia a secuencia). En el caso encoder-decoder, el decoder usa cross-attention donde Q viene del decoder y K, V del encoder.

Clase 3: Detrás de la IA — Tokenizers

La tokenización está en el origen de muchos comportamientos anómalos de los LLMs. La unidad mínima es el **token**, no el carácter, y esto explica por qué los modelos no deletrean bien, hacen aritmética inconsistente, rinden peor en idiomas distintos al inglés (más tokens por palabra), y se comportan erráticamente con tokens raros que el tokenizer vio pero el modelo apenas entrenó (SolidGoldMagikarp). Hasta el formato importa: JSON consume más tokens que YAML.

El camino del texto al token pasa por **Unicode** (que asigna un code point a cada carácter) y **UTF-8** (que reescribe cada code point como 1 a 4 bytes). Usar los 256 bytes como vocabulario daría secuencias demasiado largas, con costo de atención $O(T^2)$. **Byte-Pair Encoding** (BPE) resuelve esto: partiendo de los 256 bytes, fusiona repetidamente el par adyacente más frecuente en un token nuevo, hasta llegar a un tamaño de vocabulario deseado $V = 256 + \text{merges}$. El entrenamiento es puro conteo de frecuencias, sin gradientes. Produce dos estructuras: una tabla de merges (par $\rightarrow$ id nuevo, en orden de prioridad) y un vocabulario (id $\rightarrow$ bytes). `encode` aplica los merges por prioridad (no por frecuencia), `decode` hace el lookup inverso, y se garantiza round-trip exacto.

En la práctica, un patrón regex corta el texto en fragmentos (palabras, números, puntuación) **antes** de BPE para que los merges nunca crucen límites de palabra. GPT-4 mejora sobre GPT-2 con contracciones case-insensitive, números en grupos de 1–3 dígitos y mejor manejo de espacios. Los **tokens especiales** (`<|endoftext|>`, `<|im_start|>`, `<|im_end|>`) se agregan a mano por fuera de BPE, y el formato ChatML los usa para delimitar roles en la conversación.

Además de BPE (usado por GPT), existen **WordPiece** (BERT, fusiona por máxima verosimilitud en vez de frecuencia) y **Unigram** (T5, enfoque top-down que arranca con un vocabulario grande y poda lo que menos aporta usando EM y Viterbi).

El notebook de la clase implementa un tokenizer BPE mínimo desde cero (con `get_stats` y `merge`), verifica round-trip exacto, y luego explora tokenizers reales con tiktoken y AutoTokenizer, observando rarezas como `strawberry` → `[st, raw, berry]`, números partidos inconsistentemente y más tokens para japonés que inglés. Un segundo notebook introductorio usa Qwen3 8B con Ollama para mostrar que un LLM **solo predice el próximo token** y que todo lo demás (memoria, personalidad, herramientas, RAG) es software alrededor del predictor. Se exploran temperatura, chat templates, function calling, alucinaciones y la diferencia entre modelo base e instruct.

Clase 4: Embeddings

La tabla de embeddings de un LLM (`nn.Embedding`) es un parámetro más, entrenado end-to-end con la loss de predecir el próximo token. Los modelos que se ven en esta clase (Word2Vec, GloVe) sirven para entender el mecanismo. La métrica central es la **similitud coseno**: $\cos(\mathbf{a}, \mathbf{b}) = \mathbf{a} \cdot \mathbf{b} / (\|\mathbf{a}\| \|\mathbf{b}\|)$, que compara dirección ignorando magnitud. Se usa coseno en vez de distancia euclidiana porque la magnitud correlaciona con frecuencia y queremos factorizarla afuera.

Los **one-hot** son el punto de partida: vectores gigantes con un único 1, donde todo es ortogonal a todo y no hay noción de similitud. La **hipótesis distribucional** ("una palabra se conoce por la compañía que mantiene") sugiere usar co-ocurrencias. La **matriz de co-ocurrencia** con PPMI (Positive Pointwise Mutual Information) y SVD produce embeddings donde emerge geometría por temas, pero es ruidosa y no escala. **Word2Vec** (skip-gram) inventa una tarea de predicción descartable — dada una palabra central, predecir sus vecinas — y usa **negative sampling** para reducir el costo de $O(V)$ a $O(k)$. El resultado son vectores donde las analogías funcionan como aritmética. **GloVe** combina conteo global con aprendizaje: ajusta un modelo bilineal (sin capas, sin no linealidad) sobre la matriz de conteos. Levy & Goldberg (2014) mostraron que los tres métodos factorizan implícitamente la misma matriz de asociación. El techo de todos estos modelos es que producen un vector fijo por palabra, ciego al contexto: "bank" tiene dos sentidos pero un solo vector. La resolución llega con los embeddings **contextuales** del Transformer.

Para aplicaciones reales se necesitan **sentence embeddings**: un vector de largo fijo por oración. Promediar word vectors funciona sorprendentemente bien como piso, pero ignora el orden ("the dog bit the man" y "the man bit the dog" dan el mismo vector) y diluye oraciones largas. BERT produce vectores contextuales por token pero **no sirve para coseno** tal cual: nunca fue entrenado para que similitud semántica ⇒ cercanía en coseno. **SBERT** (Sentence-

BERT) resuelve esto con entrenamiento **contrastivo** en arquitectura siamesa: el mismo encoder embebe dos oraciones y la loss acerca las parecidas y aleja las distintas. Los embedders basados en decoder (GPT) siguen la misma receta pero leen el último token en vez del [CLS].

La **búsqueda semántica** es la "R" de RAG: query y documentos se embeben con el **mismo modelo**, se comparan por coseno y se toman los top-k. Es crítico que query y documentos usen el mismo embedder, porque el coseno solo tiene sentido dentro de un mismo espacio vectorial. Un sistema RAG tiene dos modelos: el embedder (que indexa y busca) y el LLM generador (que escribe la respuesta), y se comunican por texto. Para embedders en 2026, el benchmark de referencia es MTEB, y hay opciones pagas (OpenAI, Cohere) y open source competitivas (Qwen3-Embedding, BGE-M3, nomic-embed-text).

Los notebooks implementan coseno desde cero, co-ocurrencia + PPMI + SVD sobre Reuters, Word2Vec con gensim, GloVe pre-entrenado (incluyendo detección de sesgos como doctor$-$man$+$woman $\approx$ nurse), SBERT para sentence embeddings, búsqueda semántica verificando la invariante de mismo modelo, y un RAG mínimo con Ollama donde sin RAG el modelo inventa fechas y con RAG responde desde la base.

Clase 5: Pretraining y Fine-tuning

El entrenamiento de un LLM tiene tres etapas: el **preentrenamiento** construye una masa gigantesca de patrones (competencia), el **SFT** le da forma de asistente (comportamiento), y el **RLHF** es la capa final de ajuste. Casi todo el cómputo se gasta en el preentrenamiento.

Los datos se curan con un pipeline empírico de 8 pasos: filtrado por URL, extracción de texto limpio, detección de idioma, heurísticas de calidad (filtros Gopher y C4), deduplicación con MinHash, filtros ad-hoc y remoción de PII. Cada decisión de curación se valida con **ablations**: experimentos controlados sobre modelos chicos donde se cambia un solo factor y se mide accuracy en tareas con respuesta conocida. La lección central es que calidad $>$ cantidad: filtrar mejor rinde más que agregar más data cruda. La **perplexity** (PPL $=$ exp(cross-entropy)) mide fluidez pero no competencia real.

Las **scaling laws** muestran que la loss baja predeciblemente (power law en log-log) al escalar parámetros N, datos D o cómputo $C \approx 6ND$. **Chinchilla** (DeepMind, 2022) estableció que el óptimo está en $\sim$20 tokens por parámetro, pero el giro moderno es sobre-entrenar para abaratar la inferencia: Llama 3 8B usa $\sim$1900 tokens por parámetro. El producto del pretraining es un "simulador de documentos de internet": competencia amplia pero comportamiento crudo (ante una pregunta, puede generar más preguntas en vez de responder). El conocimiento ya está; falta activarlo. Si se necesita conocimiento nuevo de dominio, se hace **continued pretraining** (texto plano, mucho dato, caro); si se necesita cambiar comportamiento, se hace **fine-tuning** (pares supervisados, poco dato, barato).

El **instruction tuning** (SFT) entrena sobre pares instrucción $\rightarrow$ respuesta ideal. Generaliza porque la variedad de tareas enseña la habilidad de seguir instrucciones, no un catálogo fijo. LIMA demostró que mil ejemplos buenos y variados le ganan a un set enorme y ruidoso. La loss se calcula solo sobre los tokens de la respuesta (masking del prompt), y los roles se serializan con un chat template (ChatML). Full fine-tuning mueve todos los pesos y requiere $\sim$16 bytes por parámetro (un 7B $\rightarrow$ $\sim$112 GB), lo que motiva los métodos **PEFT** (Parameter-Efficient

Fine-Tuning) que congelan el base y entrenan $< 1\,\%$ de parámetros.

LoRA (Low-Rank Adaptation) explota que el ΔW del fine-tuning tiene rango bajo: en vez de aprender $\Delta W \in \mathbb{R}^{d\times d}$, aprende $B \cdot A$ con $r \ll d$. Para una capa 4096×4096 con $r = 8$, esto pasa de 16.7M a 65.5K parámetros ($\approx 0{,}4\,\%$). Se inicializa B en ceros para que el modelo arranque idéntico al base, y en inferencia se fusiona sin overhead. **QLoRA** va un paso más: cuantiza el base congelado a 4 bits (NF4, 16 niveles en los cuantiles de una normal) con double quantization y paged optimizers, permitiendo fine-tuning de un 7B en una T4 de 16 GB casi sin costo de calidad.

El notebook implementa el arco completo sobre SmolLM2-1.7B: carga del base en 4-bit ($\sim$1.0 GB), verificación de que el base no se comporta como asistente, definición manual del chat template ChatML, implementación de LoRA a mano (18 líneas) y con la librería `peft` (3.1M parámetros entrenables = $0.18\,\%$), entrenamiento de 200 pasos donde la loss baja de $\sim$3.5 a $\sim$2.3, y verificación de que el modelo ahora responde y frena correctamente. El adapter guardado pesa 6.1 MB vs 3.42 GB del modelo completo.

Clase 6: Alineamiento y Evaluación

Usando la analogía de Karpathy: pretraining es leer el libro (exposición), SFT es estudiar problemas resueltos (imitación), y RL es resolver problemas de práctica (probar y corregir). En **dominios verificables** (matemática, código), el modelo genera muchas soluciones, un verificador filtra las correctas, y se refuerzan las estrategias que llegan más seguido a la respuesta. El **razonamiento emerge** de este proceso: al optimizar por acertar, las respuestas se alargan solas porque "pensar en voz alta" ayuda. Esto es lo que produce modelos como R1, o1 y Qwen3 think, y puede superar lo que cualquier humano usaría como estrategia (como el movimiento 37 de AlphaGo).

En **dominios no verificables** ("un chiste sobre pelícanos", "resumí esto") no hay respuesta correcta. La solución es **comparar**: mostrar varias respuestas a humanos y pedirles que las ordenen, lo cual es más fácil que puntuar. Un **reward model** aprende a ser juez: mismo Transformer con una cabeza escalar en vez de vocabulario, entrenado con Bradley-Terry sobre pares (chosen, rejected). En **RLHF**, el LLM genera, el reward model puntúa, y se empuja hacia puntajes altos, pero con un **freno KL** contra una copia congelada del SFT para evitar **reward hacking** (el RL encuentra agujeros del reward model, como secuencias sin sentido con puntaje altísimo). **DPO** logra el mismo objetivo pero sin reward model y sin PPO, colapsando las dos redes en una sola loss sobre los pares. **Constitutional AI** (Anthropic) reemplaza al humano por principios en lenguaje natural y un modelo que juzga: el humano sube un nivel, de etiquetar casos a escribir principios.

En cuanto a **evaluación**, los **benchmarks** (MMLU, HellaSwag, GSM8K, HumanEval) son baterías fijas con respuesta conocida que miden capacidad pero no preferencia, y sufren de saturación y contaminación. La **evaluación por preferencia** usa el mismo mecanismo de comparación: **LLM-as-a-judge** (un modelo fuerte elige entre dos respuestas, con sesgos de posición y verbosidad) y **Chatbot Arena** (batallas ciegas votadas por humanos, ranking Elo). La unificación es notable: Elo = Bradley-Terry = la misma σ que entrena el reward model y que usa DPO.

Los principales **modos de falla** son las **alucinaciones** (el modelo inventa con confianza

porque el pretraining premia fluidez, no verdad; se mitigan con RAG, tool use y entrenando el "no sé"), los **sesgos** (heredados del corpus, medibles con tests de perturbación), y **Goodhart/reward hacking** ("cuando una medida se convierte en objetivo, deja de ser buena medida" — aparece tanto en entrenamiento como en evaluación). La única evaluación que Goodhart no rompe es la verificable. En **responsible AI**, la taxonomía de Weidinger et al. (2022) identifica 21 riesgos en 6 áreas, y los **guardrails** funcionan como Constitutional AI en inferencia: un segundo pase que verifica si la respuesta es segura antes de devolverla. La tensión de frontera es que, a medida que los modelos superan la capacidad humana de juzgar, evaluar se convuelve el problema difícil (scalable oversight).